

CONTINUUM

Verifiable Semantic Recovery for Long-Running AI Agents

The Universal Adapter for AI Agent Infrastructure

One Log. Any Harness. Zero Duplicate Work.

INVESTOR PITCH DECK | 2026

Version 0.1.2 | Apache 2.0 License | Python 3.11+

Repository: <https://github.com/Cyrax321/CONTINUUM.git>

PyPI: `pip install continuum-agent`

Website: <https://continuum-nu-six.vercel.app/>

The Problem

Modern AI agents run long tasks: hundreds of LLM calls, tool invocations, file and database writes. When they crash, the industry default is to replay everything from scratch. This causes:

- Duplicate work: The same documents re-analyzed, the same API calls re-made.
- Duplicate side effects: Invoices sent twice, emails fired multiple times, records overwritten.
- Wasted tokens and cost: Full replay burns the entire token budget again.
- Lost decisions: Reasoning and plan progress from hours of work vanish.
- No trust boundary: Agent-reported progress is taken at face value with no verification.

Existing solutions (Temporal, LangGraph, ACRFence) address parts of this problem but none provide a portable verification substrate that answers the critical question: given the state at time T and the world as it is now, is it still safe to continue?

The Solution: CONTINUUM

CONTINUUM is a verifiable semantic recovery layer for long-running AI agents. It is the universal adapter that sits beneath any AI agent framework and provides durable task state, idempotent side-effect tracking, environment revalidation, and provenance-aware trust.

Core Invariant

"Every fact carries its origin, and trust is earned, never assumed."

Three Guarantees

No self-certification: Agent-reported state is marked `REQUIRES_REVIEW` until a human confirms. Only trusted writers produce `DETERMINISTIC` state.

Side effects require claims: Every external effect is claimed in an idempotent ledger before it fires. Unclaimed effects are blocked, duplicates refused, uncertain outcomes raised for reconciliation.

Recovery verifies against reality: Resume checks file digests, dependency versions, and model identity before saying safe. Staleness propagates through the dependency graph.

How It Works

CONTINUUM separates LLM context (temporary) from durable task state (permanent). Instead of saving conversation history, it constructs a semantic checkpoint: the minimum verified information required to continue. On resume, every component is independently verified against the current environment.

Complete Feature Set

All features below are implemented, tested, and shipped in the current codebase.

Core Data Layer

- Hash-chained event log (52 event types) with integrity verification and tamper evidence

- Semantic state projection: pure fold over event prefix, reproducible and prefix-closed

- State validation with staleness propagation through dependency graph

- State diffing and version pinning with SHA-256 constraint hashes

- Provenance tracking: Origin enum (DETERMINISTIC, HUMAN, LLM, EXTERNAL_AGENT, IMPORTED, EXTERNAL_MONITOR)

Action Ledger

- Idempotent claim/complete two-phase protocol with UnknownSideEffect detection

- Idempotency key derivation with path canonicalization and token-based identity fallback

- Cross-run action index for indexed idempotency lookups

- Retry budgets with per-action-type attempt caps enforced at claim time

- Consumed-grant tracking: single-use authority references marked spent, GRANT_DENIED on reuse

- Authority reconciliation with external probe validation

- Compensating transactions with ACTION_COMPENSATED events

Recovery Engine

- Seven recovery modes: RESUME, REPAIR_AND_RESUME, REPLAN, WAIT, REQUEST_HUMAN, ROLLBACK, ABORT

- Deterministic, integrity-sealed, versioned recovery contract

- Repair planning with executable repair steps

- Family rollup: parent/child hierarchy with worst-state composition

- Fork semantics: divergent continuations branch into child runs

- Informed retry: engine-authored failure summaries injected into resumes

- Briefing curation: provenance-labeled resume context with quarantine

- Risk policy ingestion and risk-to-mode mapping

- Liveness watchdog with silence detection

- Recovery ledger: append-only hash-chained audit of recovery decisions

- Webhook notifications with HMAC-SHA256 signed delivery

Checkpointing

- Policy-driven checkpoints: manual, interval, event, semantic, context-pressure, hybrid

- Checkpoint manager with integrity sealing and RECOVERY anchors

- Bounded recovery context with protected sections

- Atomic rewind with content-addressed snapshot store and dual-state rewind

- Log compaction: pre-anchor prefix archived, live log bounded for month-long runs

Gate and Enforcement

- Pre-tool-use gate: allow/deny against ledger claims with .continuum/gate.json config

- Enforcing HTTP gateway: claim-before-fire proxy for any language, fail-closed

Replay guard: evaluate, protected_call, langgraph_protected_node

Observation hooks: auto-checkpoint, file-derived progress, async variants

MCP Server (13 Tools)

13 tools over stdio: 3 read-only + 10 mutating, deny-by-default authorization

Tools: record_progress, checkpoint, validate, resume, intercept_action, complete_action, fail_action, reconcile_action, list_actions, confirm, compensate_action, record_plan, providers

4-tier authorization policy resolution with token auth (CONTINUUM_MCP_TOKEN)

Per-caller secrets via CONTINUUM_MCP_CLIENT_TOKENS

Cross-platform MCP install/remove in .mcp.json

CLI and Interfaces

49 argparse commands with exit codes as safety contract

Only verified-safe runs exit 0: continuum resume && ./start-agent.sh cannot launch onto stale state

TUI with tree view and recovery view

Web dashboard with HITL surface (confirm/reconcile/complete buttons)

OpenTelemetry bridge: span processor turning OTel spans into evidence, zero code changes

Framework Adapters (9+)

Generic Python (GenericAgentAdapter): production-ready, trusted writes

Filesystem sandbox, Python in-process, Container (Docker), Browser (Playwright), Kubernetes

OpenAI Agents SDK: experimental, live-model tested

LangGraph: experimental, live-model tested, native BaseCheckpointSaver implementation

LangChain: experimental, live-model tested

Thin hooks (SDK-free): CrewAI, AutoGen, Pydantic AI

Security

Ed25519 attestation: continuum attest signs chain head for external verifier

Stable hashing with canonical sanitization

Trust boundary enforcement via trust gate

Lineage tokens for token-based lineage verification

Constraint validation with ID/charset validation

Benchmarks and Testing

CONTINUUM-Bench: 5 crash scenarios + argument drift + 14-scenario recovery suite + 7 fault injection

Horizon benchmark: 5 scenarios, published accuracy 0.8

Nightly bench CI with --publish

~2,864 tests collected (~2,825 passing) on Python 3.11/3.12/3.13

132 source modules, 187 test files

Architecture: Universal Adapter, One Log, Any Harness

Any harness plugs into the same hash-chained log. The same run can be written by Claude Code, resumed by LangGraph, inspected by the CLI, and approved on the dashboard. No framework cooperation is required.

Five Integration Seams

1. In-process

GenericAgentAdapter.intercept_action() and wrap_tool() on LangChain, LangGraph, OpenAI Agents SDK. Python frameworks, trusted writes.

2. MCP Server

continuum-mcp: 13 tools over stdio. Any MCP-capable client. 3 read-only + 10 mutating, allowlist via CONTINUUM_MCP_MUTATING_CLIENTS.

3. CLI Lifecycle Hooks

continuum hooks install claude-code --with-gate (also gemini, codex). SessionStart briefing, PostToolUse observe, PreToolUse gate. No CLAUDE.md needed.

4. Enforcing HTTP Gateway

continuum gateway --port 8765 with .continuum/gateway.json. Any language, any outbound HTTP must have a claim. Gateway settles from real status code.

5. OpenTelemetry Bridge

make_span_processor(storage). Any traced app, spans become TOOL_COMPLETED evidence.

Storage Architecture (Schema v6)

events: Hash-chained append-only log (52 event types)
 runs: Run metadata with parent_run_id for multi-agent
 versions: SemanticState snapshots per checkpoint
 checkpoints: Sealed checkpoint records with RECOVERY anchors
 action_index: Cross-run idempotency projection: indexed reads, not full scans
 events_archive: Compacted prefix storage: continuum compact bounds live log for weeks
 lg_checkpoints / lg_writes: LangGraph native persistence via make_continuum_checkpointinter(storage)

Recovery Decision Tree

RESUME < REPAIR_AND_RESUME < REPLAN < WAIT < REQUEST_HUMAN < ROLLBACK < ABORT

The engine takes the maximum severity across all signals, so safety never loses to convenience. Every continuum resume returns a sealed contract with recovery status, verified/invalidated components, executable human_steps, and family aggregation for multi-agent runs.

Traction and Metrics

All numbers below are from real test output, not projections.

2,864 Tests Collected	132 Source Modules	187 Test Files	Duplicate
0 Duplicate Work	0.8 Horizon Accuracy	7/7 E2E Autonomy Score	3 Cont

Benchmark Comparison

CONTINUUM: 0 duplicate side effects, 0 duplicate work
 Naive replay: 50 duplicates for 50 actions attempted twice
 Fault injection: 7 scenarios, detection rate 1.0, unsafe resume rate 0.0

Live Model Validation (gpt-4o-mini via OpenRouter)

LangChain: Soft resume PASS (1 side effect, resume safe). Hard crash PASS (request_human, 1 uncertain).
 OpenAI SDK: Soft resume PASS (1 side effect, request_human). Hard crash PASS (request_human, 1 uncertain).
 LangGraph: Soft resume PASS (1 side effect, resume safe). Hard crash PASS (request_human, 1 uncertain).

E2E Autonomy Test

Three full Claude Code sessions (Opus 4.8), each hard-killed mid-batch and resumed in a brand-new session. All scored 7/7 on mechanics checks. Agents autonomously called record_progress, routed sends through intercept_action -> write -> complete_action, called resume before acting, and refused to re-send invoices they verified as already sent.

Distribution

PyPI: pip install continuum-agent (v0.1.2)
 Homebrew: brew install wized2/continuum/continuum
 Docker: docker run --rm ghcr.io/cyrax321/continuum
 CI: Green on ruff, ruff format, mypy strict, and tests across Python 3.11/3.12/3.13
 Published in 6 languages: English, Chinese, Spanish, Japanese, Portuguese, Korean

Market Opportunity

The AI agent infrastructure market is projected to reach \$47B by 2030 (Grand View Research). As agents move from demos to production, reliability and recovery become critical requirements. CONTINUUM addresses the verification and recovery layer that every production agent stack needs.

Target Customers

- AI agent startups needing production-grade reliability
- Enterprise AI teams running long-running agent workflows
- Agent framework developers seeking a recovery layer
- DevOps/SRE teams managing agent infrastructure

Competitive Moat

- Framework-agnostic: Works with any agent framework via 5 integration seams
- Provenance-aware: No competitor prevents agent self-certification
- Deterministic contracts: Recovery decisions are byte-identical for the same inputs
- Tamper-evident: Hash-chained log with Ed25519 attestation
- Battle-tested: Real process kills, live model validation, adversarial MCP audit

Roadmap

Completed (Phases 1-15)

Data models, semantic state, persistence, checkpointing, validation, action ledger, recovery engine, CLI
Framework adapters (Generic, OpenAI, LangGraph, LangChain, CrewAI, AutoGen, Pydantic AI)
MCP server with 13 tools, authorization, and caller authentication
Enforcing gate, HTTP gateway, OpenTelemetry bridge, observation hooks
Dashboard, TUI, benchmark suite, log compaction, fork semantics, informed retry
Ed25519 attestation, consumed-grant tracking, HITL surface

Planned (Next)

Cloud API (Phase 13): FastAPI + PostgreSQL hosted multi-tenant service (Postgres backend and HTTP sidecar transport already shipped)
Months-scale durability plane: milestone-anchored plans, structured attempt memory, atomic dual-state rewind
Public recovery-correctness benchmark
Large payload offloading
Weeks-scale benchmark with token cost table

Team

Anandhu P Shaji (@Cyrax321)

Creator and maintainer of CONTINUUM. Built the project from a single invariant over three weeks, then grew it with 36+ community contributors. Background in AI agent infrastructure and distributed systems.

Community

36+ contributors from around the world. Active open-source community with contributors from India, Iran, Japan, Vietnam, Korea, and Europe. Project published in 6 languages.

The Ask

CONTINUUM is seeking investment to accelerate development of the cloud API, expand the team, and capture the AI agent reliability market. The core technology is built, tested, and proven with real agents and real crashes. We are ready to scale.

Use of Funds

- Cloud API development and hosted infrastructure
- Engineering team expansion
- Enterprise features and support
- Community growth and ecosystem development

Get In Touch

Repository: <https://github.com/Cyrax321/CONTINUUM.git>

PyPI: `pip install continuum-agent`

Website: <https://continuum-nu-six.vercel.app/>

GitHub: <https://github.com/Cyrax321/CONTINUUM>

Apache 2.0 License. Open source. Community driven.

Appendix: Honest Limitations

CONTINUUM is transparent about what it does not do. These are documented in the codebase:

- Gate does not see inside shell commands (Bash/curl bypass structured tool claims)
- Postgres backend is CI-tested but not battle-tested in production
- One level of multi-agent hierarchy in v1
- Framework adapters remain experimental (prefer GenericAgentAdapter for production)
- Agent/MCP runs need explicit confirm before auto-resume (by design)
- Large payload offloading not yet implemented
- Cloud API (Phase 13) not yet started (Postgres backend and HTTP sidecar shipped)

Appendix: What CONTINUUM Is Not

- Not an LLM: A reliability layer for agents that use LLMs
- Not an agent framework: A recovery layer that plugs into any framework
- Not a vector database: Structured semantic state, not embeddings
- Not a RAG system: Verified checkpoints, not retrieval-augmented memory
- Not a workflow engine: A recovery layer, not an orchestrator

Appendix: Related Work

CONTINUUM sits at the overlap of durable execution, idempotent side-effect tracking, and crash recovery for LLM agents. The closest neighbors are machine-checked resume contracts (Khan 2026), agentic transaction processing with constraint-gated admission (Mnemosyne 2026), checkpoint-rollback attack analysis (ACRFence 2026), and design-level prompt-injection defense (CaMeL 2025).